

RAG from Zero to Hero

NIK ABDUL AQMAL BIN ZUNAIDI

Table of contents

0.1 Data Ingestion	1
1 WE HAVE COMPLETE OUR DATA INGESTON SECTION	6
1.1 lets move to the chunking section	6
2 RAG PIPELINES DATA INGESTION TO VECTOR DB PIPELINE	6

0.1 Data Ingestion

We have several loaders in LangChain

Retrieval Augmented Generation: Process of optimizing LLM output
so it will refer to authorized knowledge base outside its training data source

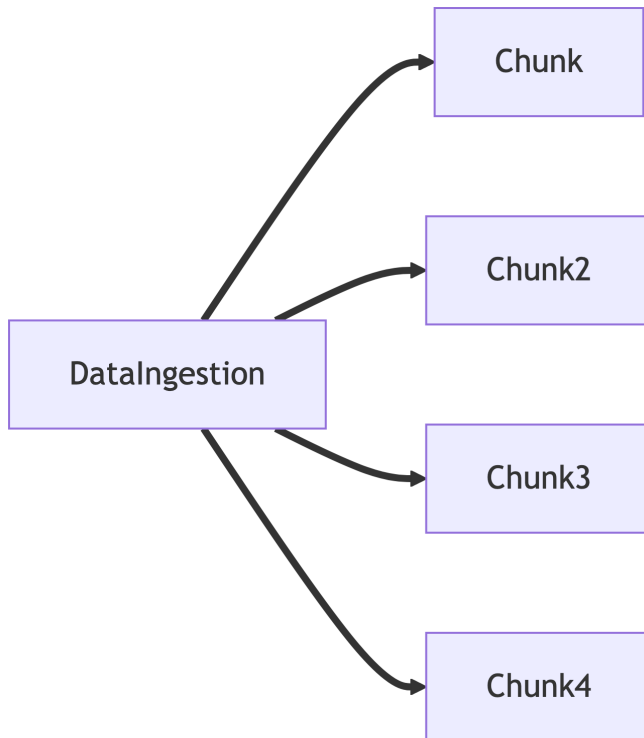
```
from langchain_core.documents import Document
```

getting the query from user and do embedding operation on it and refers out vector db
RETRIEVAL PIPELINE

we have much more type of RAG so like Agentic RAG

Augmentation means we give context to llm and generate the output **Data Ingestion and Data Parsing**

```
#
```



so for every chunk we apply embeddings and finally store it into our final vector db **embedding**: convert text to vectors

```
### document structure
from langchain_core.documents import Document
# if we wanna improve our RAG we shall use metadata
doc=Document(
    page_content="this is the main text content I am using to create RAG",
    metadata={
        "source": "example.txt",
        "pages":1,
        "author":"Khrisk Naik",
        "date_created": "2025-01-07",
    }
)

doc
```

```
Document(metadata={'source': 'example.txt', 'pages': 1, 'author': 'Khrisk Naik', 'date_created': '2025-01-07'}, page_content='this is the main text content I am using to create RAG')
```

```

# create a simple txt file
import os

os.makedirs("./data/text_files", exist_ok=True)
# using one dot will create this folder in current pwd

sample_texts={
    "./data/text_files/python_intro.txt": """Python Programming Introductio

Python is a high level, interpreted programming language known for its simplicity and readability.
Created by Guido van Rossum and first released in 1991, Python has become one of the most popular
Programming languages in the world.

Key Features:
- easy to learn and use
- extensive standard Library
-

python also widely used in web development, data science, artificial intelligence, and automation.

""",
    "./data/text_files/machine_learning.txt": """Machine Learning Basics

Machine learning is a subset of artificial intelligence that enables system to learn and improve from data.

Types of Machine Learning:
1. Supervised Learning: Learning with labeled data
2. Unsupervised Learning: Finding patterns in unlabeled data
3. Reinforcement Learning: Learning through rewards and penalties

. """
}

# what does 'w' means here ??
# w - means - write
# r - means - Read
# a append
# x creates new files
# w+ write + read
# a+ append + read

for filepath, content in sample_texts.items():
    with open(filepath, 'w', encoding='utf-8') as f:

```

```

        f.write(content)

print(" Sample text files is created")

```

Sample text files is created

```

from langchain_community.document_loaders import TextLoader

# declaring the loader this was the pythonic ways
loader=TextLoader("./data/text_files/python_intro.txt", encoding="utf-8")
document= loader.load()
print(document)

```

[Document(metadata={'source': './data/text_files/python_intro.txt'}, page_content='Python Pro

```

from langchain_community.document_loaders import DirectoryLoader

## load all the text files from the Directory
dir_loader=DirectoryLoader(
    "./data/text_files",
    glob="**/*.txt", ## Pattern to match files
    loader_cls= TextLoader, ## loader class to use
    loader_kwargs={'encoding': 'utf-8'},
    show_progress=True

)

documents=dir_loader.load()
documents

```

```

0%|          | 0/2 [00:00<?, ?it/s]
100%|         | 2/2 [00:00<00:00, 4616.74it/s]

```

[Document(metadata={'source': 'data/text_files/python_intro.txt'}, page_content='Python Prog
Document(metadata={'source': 'data/text_files/machine_learning.txt'}, page_content='Machine


```
Document(metadata={'producer': 'macOS Version 26.3 (Build 25D125) Quartz PDFContext', 'creator': 'magazine.com/news/cybercrime-driven-\nfour-\ndistinct/#::~text=A%20new%20report%20from%20Malware\n13/key-\nissues/cyber-organized-crime-activities.html'})]
```

1 WE HAVE COMPLETE OUR DATA INGESTION SECTION

1.1 lets move to the chunking section

2 RAG PIPELINES DATA INGESTION TO VECTOR DB PIPELINE

```
import os
from langchain_community.document_loaders import PyPDFLoader, PyMuPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from pathlib import Path

def process_all_pdfs(pdf_directory):
    """Process all PDF files in a directory"""
    # creating list to append all document
    all_documents = []
    pdf_dir = Path(pdf_directory)

    # find all pdf files recursively
    pdf_files = list(pdf_dir.glob("**/*.pdf"))

    print(f"Found {len(pdf_files)} PDF files to process")

    for pdf_file in pdf_files:
        print(f"\nProcessing: {pdf_file.name}")
        try:
            loader = PyMuPDFLoader(str(pdf_file))
            documents = loader.load()

            # add the source of information to metadata
            for doc in documents:
                doc.metadata['source_file'] = pdf_file.name
                doc.metadata['file_type'] = 'pdf'
```


[illegible]

```
def split_documents(documents, chunk_size=1000, chunk_overlap=200):
    """splitting the documents into smaller chunks for better RAG performance"""
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        separators=["\n\n", "\n", " ", ""]
    )
    split_docs = text_splitter.split_documents(documents)
    print(f"Split {len(documents)} documents into {len(split_docs)} chunks")

    # show the example of a chunk
    if split_docs:
        print(f"\nExample chunk:")
        print(f"Content: {split_docs[0].page_content[:200]}...")
        print(f"Metadata: {split_docs[0].metadata}")

    return split_docs
```

```
chunks=split_documents(all_pdf_documents)
chunks
```

Split 22 documents into 10 chunks

Example chunk:

Content: Organized Crime and Cyberterrorism...

Metadata: {'producer': 'macOS Version 26.3 (Build 25D125) Quartz PDFContext', 'creator': 'PowerShell', 'creationdate': '2025-08-27T10:00:00', 'author': 'PowerShell', 'subject': 'PowerShell', 'title': 'PowerShell', 'keywords': ''}


```

def _load_model(self):
    """Load the SentenceTransformer model"""
    try:
        print(f"Loading embedding model: {self.model_name}")
        self.model = SentenceTransformer(self.model_name)
        print(f"Model loaded successfully. Embedding dimension: {self.model.get_embedding_dimension()}")
    except Exception as e:
        print(f"Error loading model {self.model_name}: {e}")
        raise

def generate_embeddings(self, texts: List[str]) -> np.ndarray:
    """
    Generate embeddings for a list of texts

    Args:
        texts: List of text strings to embed

    Returns:
        numpy array of embeddings with shape (len(texts), embedding_dim)
    """
    if not self.model:
        raise ValueError("Model not loaded")

    print(f"Generating embeddings for {len(texts)} texts...")
    embeddings = self.model.encode(texts, show_progress_bar=True)
    print(f"Generated embeddings with shape: {embeddings.shape}")
    return embeddings

# and then like always we need to initialize the embedding manager
embedding_manager=EmbeddingManager()
embedding_manager

```

Loading embedding model: all-MiniLM-L6-v2

```

Loading weights:  0%|          | 0/103 [00:00<?, ?it/s]
Loading weights: 100%|         | 103/103 [00:00<00:00, 4507.04it/s]

```

Model loaded successfully. Embedding dimension: 384

<__main__.EmbeddingManager at 0x3141684a0>